06-FlyWeigth

- The **Flyweight Pattern** is a way to **save memory** when you have a lot of similar objects.
- ប្រើដើម្បីធ្វើការបង្កើត object អោយអោយជាន់គ្នាច្រើនពេល ដើម្បីធ្វើការ Save Memory

🧠 Real-Life Examples

- Text editors (characters)
- Game development (trees, bullets)
- HR system (same job roles reused)
- Icons in UI (same image reused)

⚡ When to Use

Use Flyweight when:

- You have **many objects**
- Objects share **common data**
- Memory usage is high

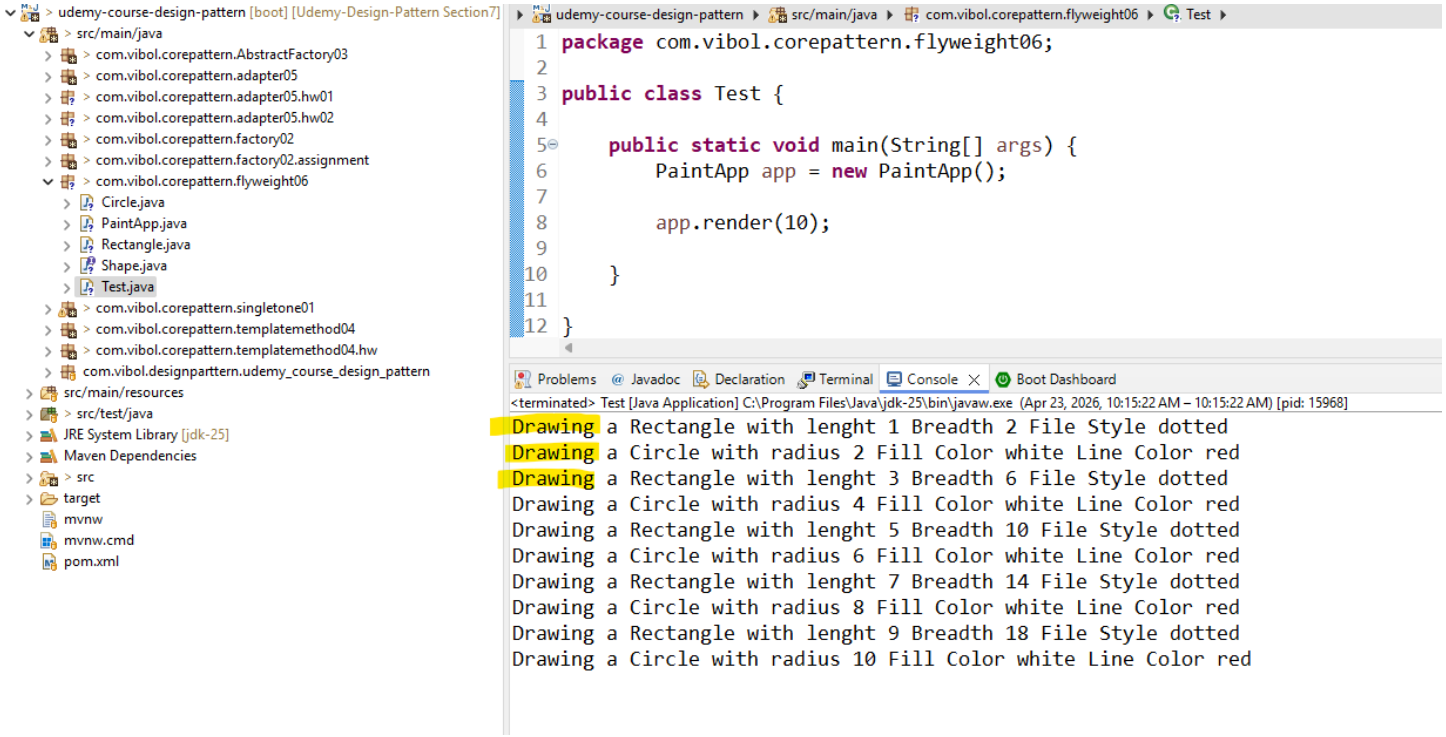
🚫 When NOT to Use

- If objects are very different
- If sharing makes code too complex

🧩 Easy Summary

👉 "Reuse instead of recreate"

Example:



វាធ្វើការបង្កើតបង្កើត Object ច្រើនចឹងធ្វើអោយ Memory ត្រូវការអស់ច្រើន

Step by Step Create Flyweight

- Separate the Extrinsic State (រកមើល Filed ចាំបាច់ត្រូវតែមានមិនអាចខ្វះបាន)- No command
- Pass them as parameters
- Creating a factory class

1- Separate the Extrinsic State

```
public class Circle implements Shape {

    private String lable;
    private int radius;
    private String fillColor;
    private String lineColor;
```

```
public class Rectangle implements Shape {

    private String lable;
    private int length;
    private int breadth;
    private String fileStyle;
```

So we can over right the method in the Interface:

udemy-course-design-pattern ▸ src/main/java ▸ com.vibol.corepattern.flyweight06 ▸ Shape ▸

```java
package com.vibol.corepattern.flyweight06;

public abstract class Shape {
    // Circle
    void draw(int radius, String fillColor, String lineColor) {

    }

    //Rectangle
    void draw(int length, int breadth, String fileStyle) {

    }

}

```

1- Circle class

*Circle.java ✕ Shape.java

udemy-course-design-pattern ▸ src/main/java ▸ com.vibol.corepattern.flyweight06 ▸ Circle ▸

```java
package com.vibol.corepattern.flyweight06;

public class Circle extends Shape {

    private String lable;

    public Circle() {
        lable = "Circle";
    }

    @Override
    public void draw(int radius, String fillColor, String lineColor) {
        System.out.println("Drawing a " + lable + " with radius " + radius
                + " Fill Color " + fillColor + " Line Color " + lineColor);
    }
}
```

2- Rectangle

Circle.java | Shape.java | Rectangle.java | Rectangle.java

udemy-course-design-pattern ▸ src/main/java ▸ com.vibol.corepattern.flyweight06 ▸ Rectangle ▸

```java
package com.vibol.corepattern.flyweight06;

public class Rectangle extends Shape {

    private String lable;

    public Rectangle() {
        lable = "Rectangle";
    }

    @Override
    public void draw(int length, int breadth, String fileStyle) {
        System.out.println("Drawing a " + lable + " with lenght " + length + " Breadth " + breadth
                + " File Style " + fileStyle);
    }

}
```

3- Shape Factory (core that we use to create Factory)

Circle.java | ShapeFactory.java

udemy-course-design-pattern ▸ src/main/java ▸ com.vibol.corepattern.flyweight06 ▸ ShapeFactory ▸ getShape(String) : Shape

```java
package com.vibol.corepattern.flyweight06;

import java.util.HashMap;

// create cache (storage)
public class ShapeFactory {

    private static Map<String, Shape> shapes = new HashMap<>();

    public static Shape getShape(String type) {
        Shape shape = null;
        if (shapes.get(type) != null) {
            shape = shapes.get(type);
        } else {
            if (type.equals("circle")) {
                shape = new Circle();
            } else if(type.equals("rectangle")) {
                shape = new Rectangle();
            }

            shapes.put(type, shape);
        }
        return shape;
    }

}
```

4- Testing

```java
package com.vibol.corepattern.flyweight06;

public class Test {

    public static void main(String[] args) {
        PaintApp app = new PaintApp();

        app.render(10);

    }

}
```

ចំងពេលដែលយើងបង្កើត Factory វាធ្វើការ Check មើលសិនបើវា មាន Object នឹងហើយវាមិនបង្កើតទៀតទេ។

```java
// create cache (storage)
public class ShapeFactory {

    private static Map<String, Shape> shapes = new HashMap<>();

    public static Shape getShape(String type) {
        Shape shape = null;
        if (shapes.get(type) != null) {
            shape = shapes.get(type);
        } else {
            if (type.equals("circle")) {
                shape = new Circle();
            } else if(type.equals("rectangle")) {
                shape = new Rectangle();
            }

            shapes.put(type, shape);
        }
        return shape;
    }

}
```